

NDTwin Network Digital Twin

# Where the Failover Time Went

— and what a sample costs

---

# Outline

---

## 0 Background

pp. 4–5

What was promised last time, and where the three data planes attach.

---

## 1 Answering the open questions

pp. 7–26

The failover budget, faster detection, what a sample costs, and a deadlock in the baseline.

---

## 2 Engineering, and what's next

pp. 28–33

Bring-up as one command, truncation and batching, and what the next report will carry.

## SECTION 0

# Background

What was promised last time, and where the three data planes attach

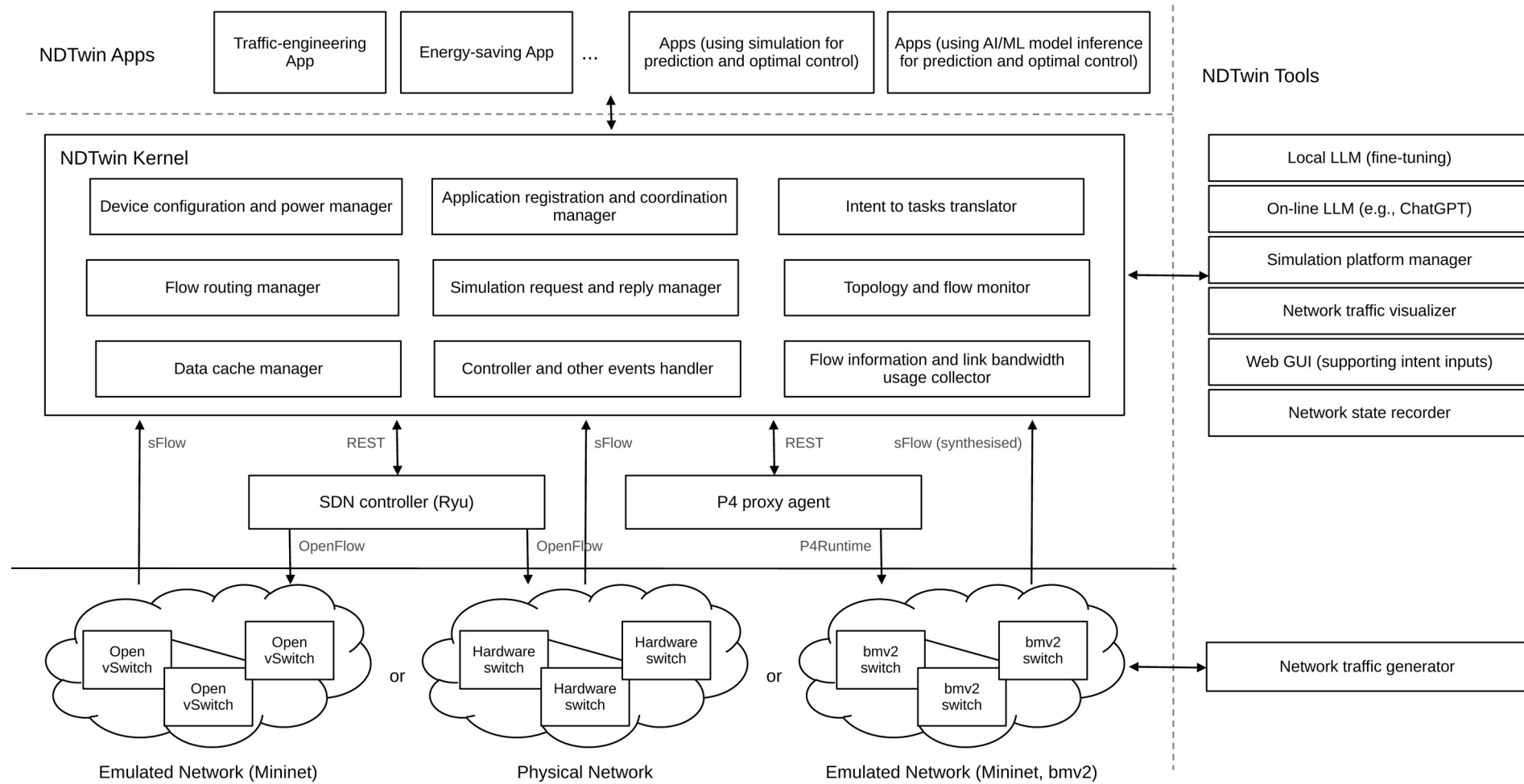
---

# Where we left off

Five things were asked for — three on the 20 August closing slide, two in the meetings since. Each came with a test

|                                                                              |                                                                          |                                                                                                            |
|------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| the test we set ourselves                                                    |                                                                          |                                                                                                            |
| 1                                                                            | <b>Find where the OVS 128-host recovery time goes</b>                    | It balances — and 87% of it is waiting to notice.                                                          |
|                                                                              | ANSWERED                                                                 |                                                                                                            |
| detection + recompute + install must add up to 51.75 s                       |                                                                          |                                                                                                            |
| 2                                                                            | <b>Make failure detection faster without making it wrong</b>             | 3.9× faster. Idle false positives: zero.                                                                   |
|                                                                              | ANSWERED                                                                 |                                                                                                            |
| judged on the false-positive rate, not on the detection time                 |                                                                          |                                                                                                            |
| 3                                                                            | <b>Promote the fast bmv2 build to the default</b>                        | The binaries can be told apart. The suite has not been run.                                                |
|                                                                              | PREREQUISITE                                                             |                                                                                                            |
| the whole LO–L4 suite must pass on the fast build                            |                                                                          |                                                                                                            |
| 4                                                                            | <b>Re-measure the sFlow jitter — baseline, and across sampling rates</b> | Inherited, not ours — and it stops buying precision at 1-in-32.                                            |
|                                                                              | ANSWERED                                                                 |                                                                                                            |
| one fabric, one flow: swap only the kernel binary, then only the sample rate |                                                                          |                                                                                                            |
| 5                                                                            | <b>Try packet truncation, and batching</b>                               | Truncation works and buys nothing. Merge is wired and verified; its effect on the ceiling is not measured. |
|                                                                              | IN PART                                                                  |                                                                                                            |
| either one moves the sampling ceiling, or the cost is not per byte           |                                                                          |                                                                                                            |

## NDTwin: A Framework for Network Digital Twins



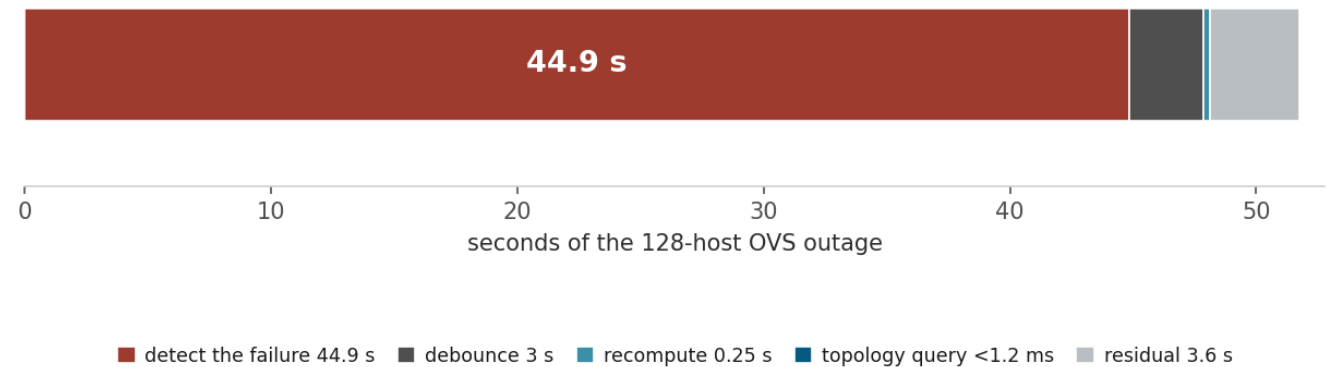
## SECTION 1

# Answering the open questions

Everything in this section was asked for — three written down on 20 August, the rest in the meetings since

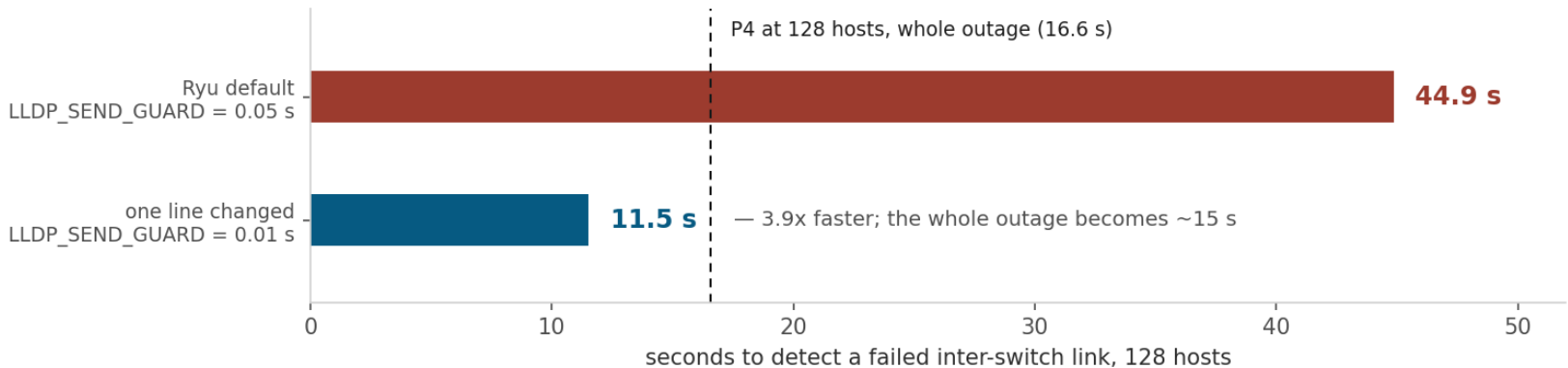
---

87% of a 128-host OVS outage is waiting to notice



The threshold is untouched -- only the interval between probes

Still six consecutive misses before a link is called dead. Lowering THAT would trade against false positives; this does not.



# Where the recompute time actually goes

The walk installs 1,280 rules and builds 16,256 path entries — only one of those grows with the square

| hosts | rules | install | path entries | report  | report % |
|-------|-------|---------|--------------|---------|----------|
| 8     | 80    | 0.007 s | 56           | 0.003 s | 30%      |
| 16    | 160   | 0.009 s | 240          | 0.009 s | 50%      |
| 32    | 320   | 0.022 s | 992          | 0.052 s | 70%      |
| 64    | 640   | 0.044 s | 4,032        | 0.286 s | 86%      |
| 128   | 1,280 | 0.103 s | 16,256       | 2.063 s | 95%      |

## 1 Installing rules is linear

16× the hosts, 14.7× the install. One rule per (switch, destination).

## 2 Building the path table is super-quadratic

The same 16× costs 688×. One entry per ordered host pair — that is what “16,256 paths” has always meant.

### One variable per cell

Same live fabric in all five; only the topology model is swapped. Ten real switches, one noise floor.

### So “routing is slow” has to be split

The bookkeeping is slow, not the programming of the switches. At 128 hosts the recompute is 0.25 s once the host lookup is indexed.

# Ruling out the controller's topology read path

The kernel re-reads three Ryu topology endpoints on every change — so all three were timed at both fabric sizes

| endpoint                | 4 hosts            | 128 hosts           | time  | bytes  |
|-------------------------|--------------------|---------------------|-------|--------|
| /v1.0/topology/switches | 0.461 ms / 4,158 B | 1.127 ms / 17,154 B | 2.44× | 4.13×  |
| /v1.0/topology/hosts    | 0.346 ms / 812 B   | 1.225 ms / 26,345 B | 3.54× | 32.44× |
| /v1.0/topology/links    | 0.449 ms / 7,176 B | 0.797 ms / 7,176 B  | 1.78× | 1.00×  |

## Why these hold

n = 20, median, first three discarded. Each cell separated by a full teardown — three different kernel pids, so nothing was inherited.

## One assertion missed what moved

A fourth endpoint was dropped: byte-identical at both sizes, because the small cell served a path table built for the large one. The probe asserted the host count — against an endpoint the path table is not derived from.

## 1 Under 1.3 ms at both sizes

Four orders of magnitude below the 51.75 s being explained.

## 2 The shape is wrong too — the stronger argument

/hosts ships 32× the data for 3.5× the time. Sub-linear serialisation, the opposite of super-linear work on a bigger graph.

# One constant, and what it did not change

Detection is set by how often a port is probed, not by how much evidence is required — only the first of those was touched

---

## 1 The probe interval

0.05 → 0.01. Ryu probes every port in turn and sleeps between sends, so the gap between two probes of one port is  $\text{port-count} \times \text{guard}$ . At 160 ports: 44.9 s, then 11.5 s.

## 2 The recompute

Host lookup indexed, cache token made  $O(1)$ . The walk goes 2.17 s → 0.25 s.

## 3 A spare lever, left off by default

Ports that never answer LLDP can be probed once every N rounds, so detection scales with switches, not ports.

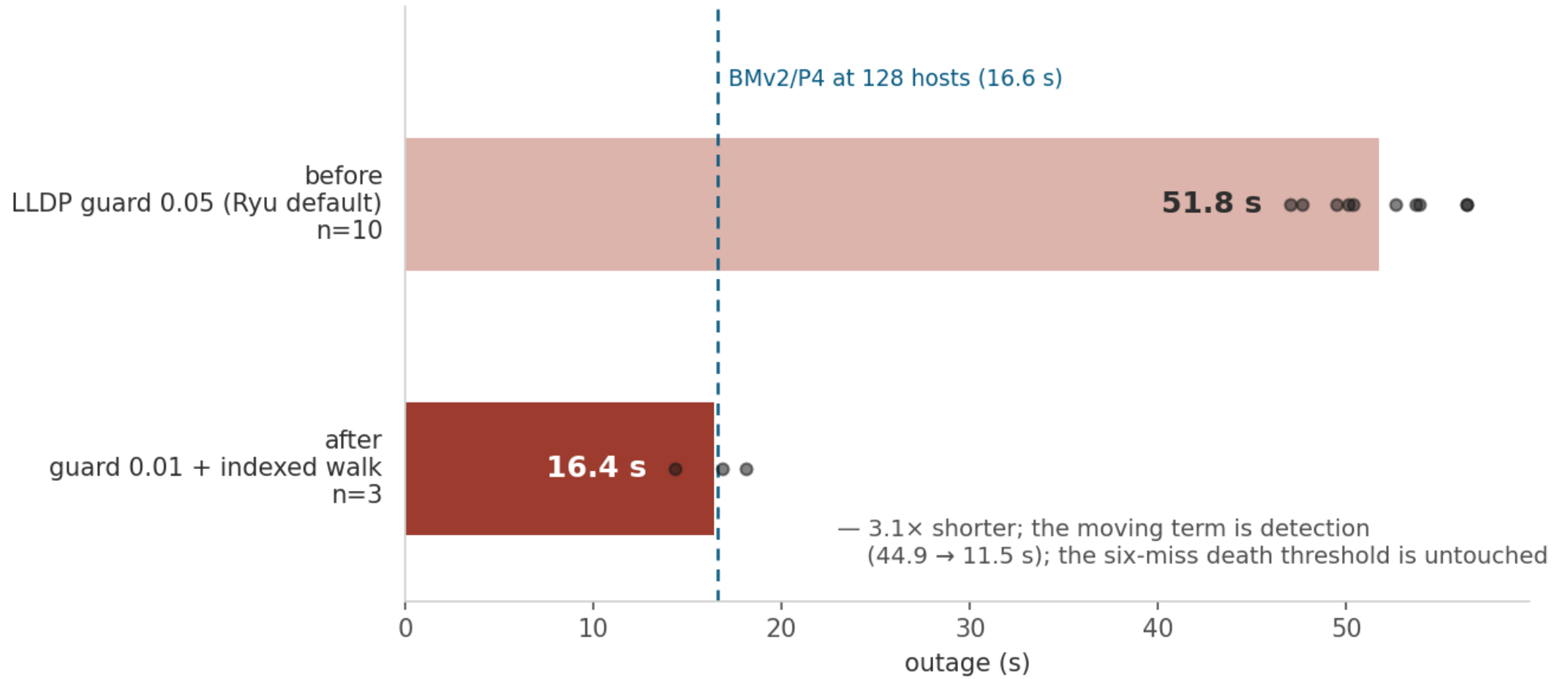
### The threshold is untouched

Still six consecutive misses before a link is called dead. The other lever — dropping that to two — reaches the same number by accepting weaker evidence. That contrast is the point of this page.

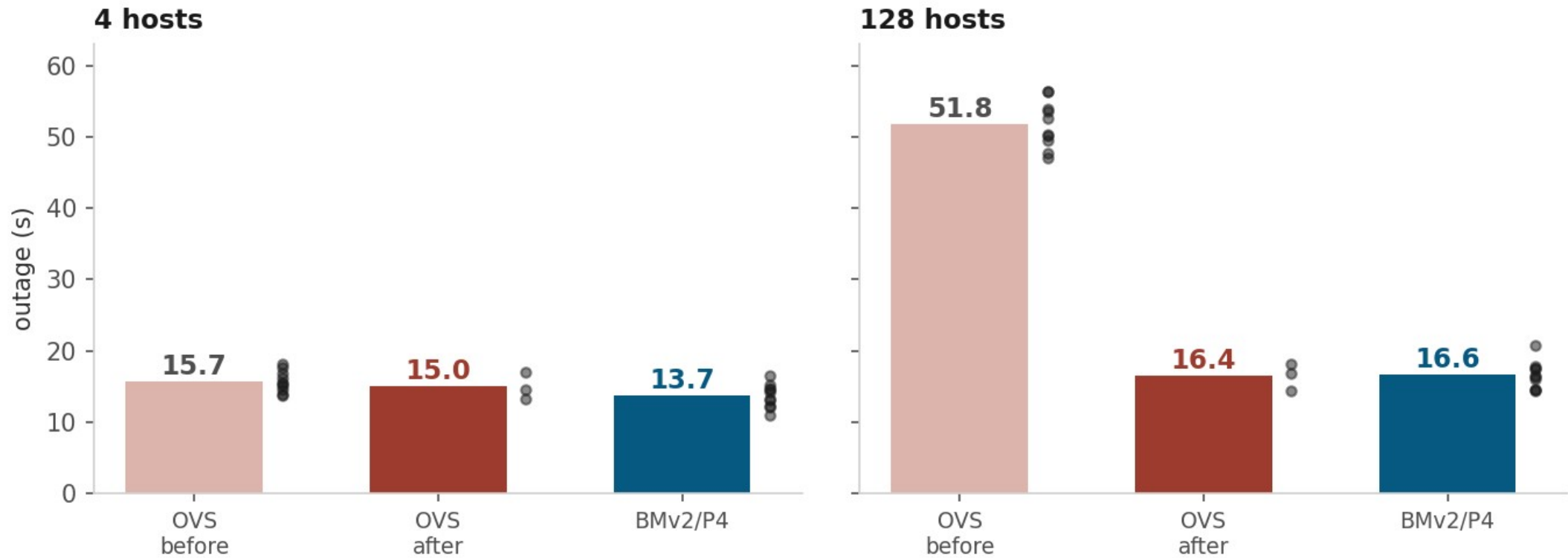
### Judged on false positives, as promised

Three cells, 20 minutes idle each, ~48,000 probe opportunities per window: zero links deleted, zero topology-changed events — and every cell still caught a real injected failure.

## The same link failure, before and after the fix



## OVS vs BMv2 after the fix: the scaling penalty is what changed



# Which bmv2 produced which number

Both builds answer --version with the same string — so this page identifies them by what actually differs

|           | stock               | fast                           |
|-----------|---------------------|--------------------------------|
| path      | /usr/local/bin/     | /usr/local/bmv2-fast/bin/      |
| --version | 1.15.3-f0b7d201     | 1.15.3-f0b7d201 ← same         |
| sha256    | 327fa7d1...         | 3ff54b5c...                    |
| size      | 9,576,568 B         | 92,147,960 B                   |
| build     | -O0 -g, logging on  | -O3 -march=native, logging off |
| measured  | ~40 Mbps / 3.6k pps | 460-530 Mbps / 50.8k pps       |

## 1 Identify by hash, not by version string

sha256, BuildID and size separate the two; --version does not.

## 2 Past -O3 numbers can be attributed backwards

The source SHA is inside the binary's own --version, and that tree is still on disk — so an earlier report calling the -O3 cell unpinnable was wrong.

### The prerequisite, not the test

The criterion was that the whole L0-L4 suite passes on the fast build. It has not been run. What exists now is the thing that had to come first — being able to say which binary produced a number.

### One seam, not two

Choosing a binary happens in one place, shared by both topologies.

## Sampling harder buys precision on a $\sqrt{\lambda}$ law

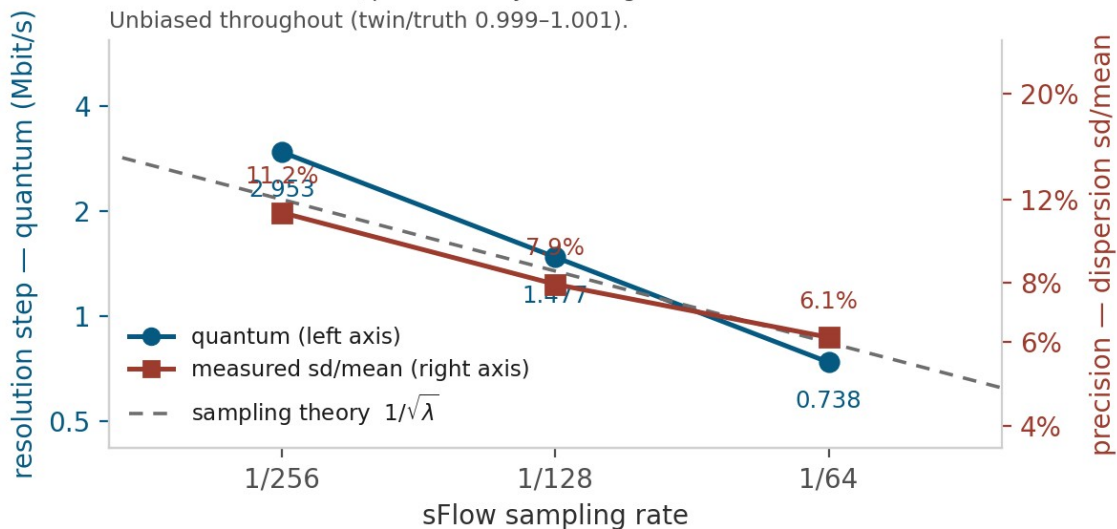
### What you get

quantum = rate  $\times$  1442 B  $\times$  8, exact at all three rates.

$\lambda = 70 / 139 / 278$  samples per 1 s window.

Resolution 4.00 $\times$  finer, precision only 1.83 $\times$  tighter.

Unbiased throughout (twin/truth 0.999–1.001).

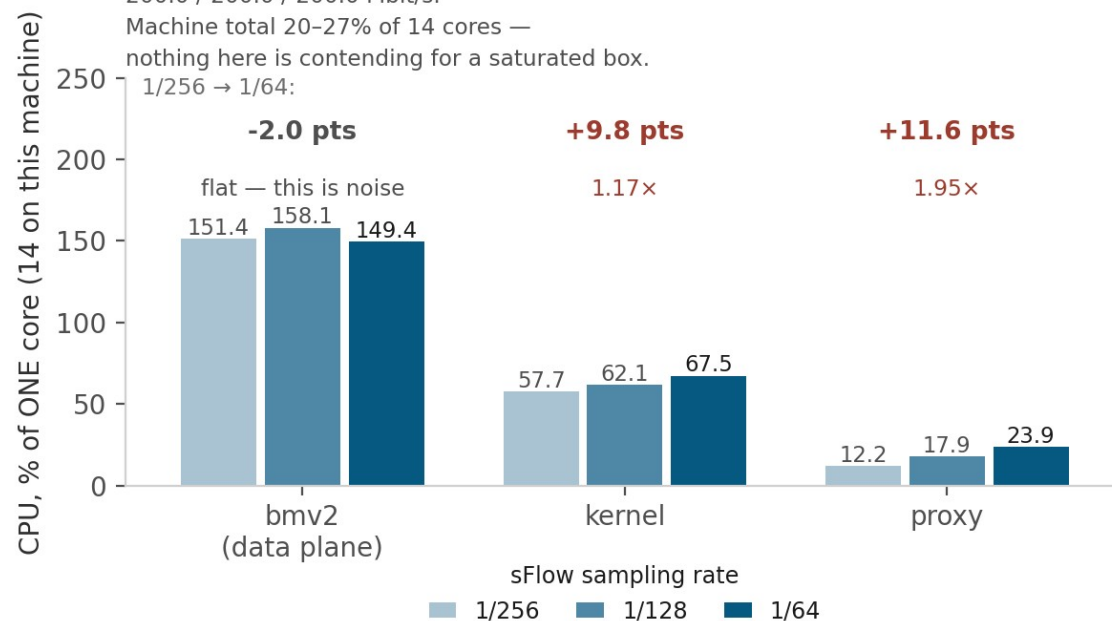


### What it costs

Delivered throughput identical at all three rates:  
200.0 / 200.0 / 200.0 Mbit/s.

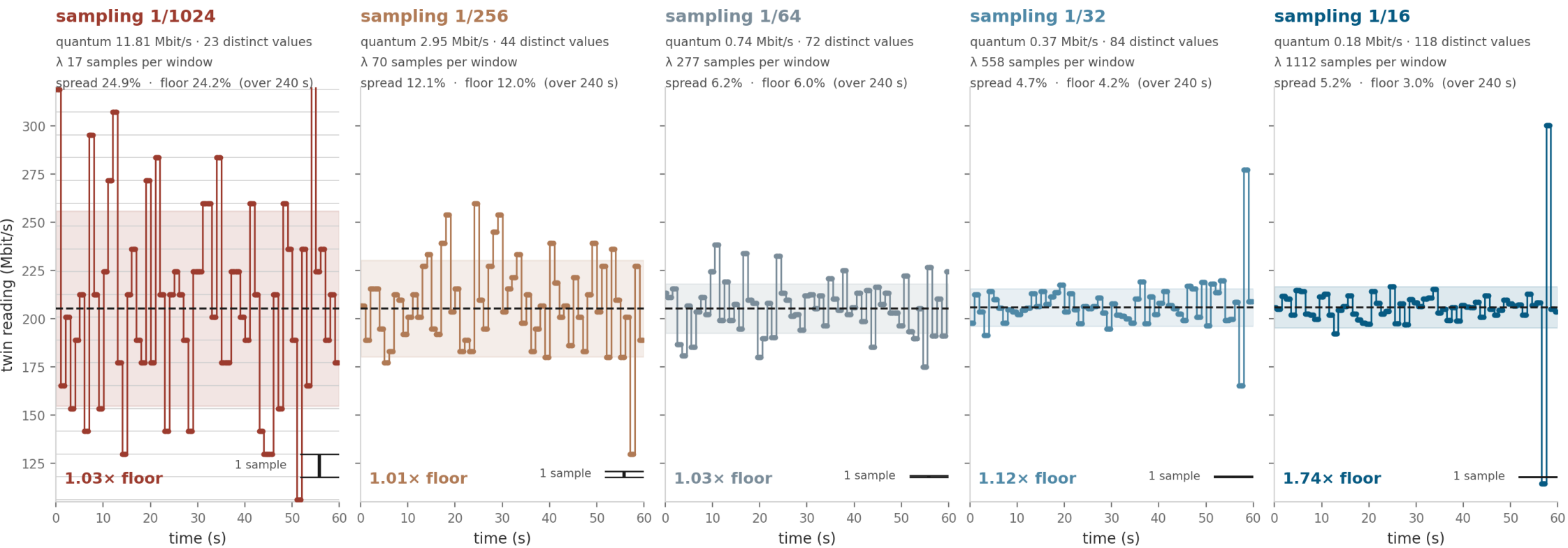
Machine total 20–27% of 14 cores —

nothing here is contending for a saturated box.



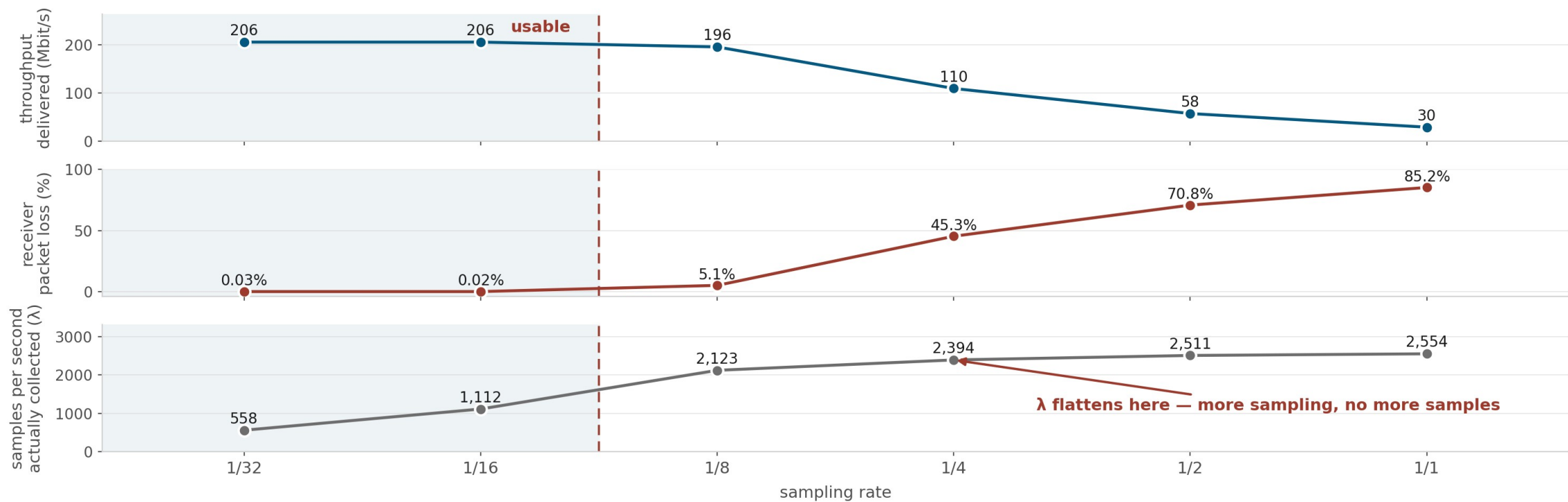
The staircase dissolves as you sample harder

One fabric, one 200 Mbit/s flow, one edge — only the pipeline's SAMPLE\_RATE differs. The quantum falls 64× from left to right, so the same true rate is reported in ever finer steps. Faint lines are the attainable values; dashed line is ground truth from the kernel's NIC counters; band is ±1 sd over the full 240 s.



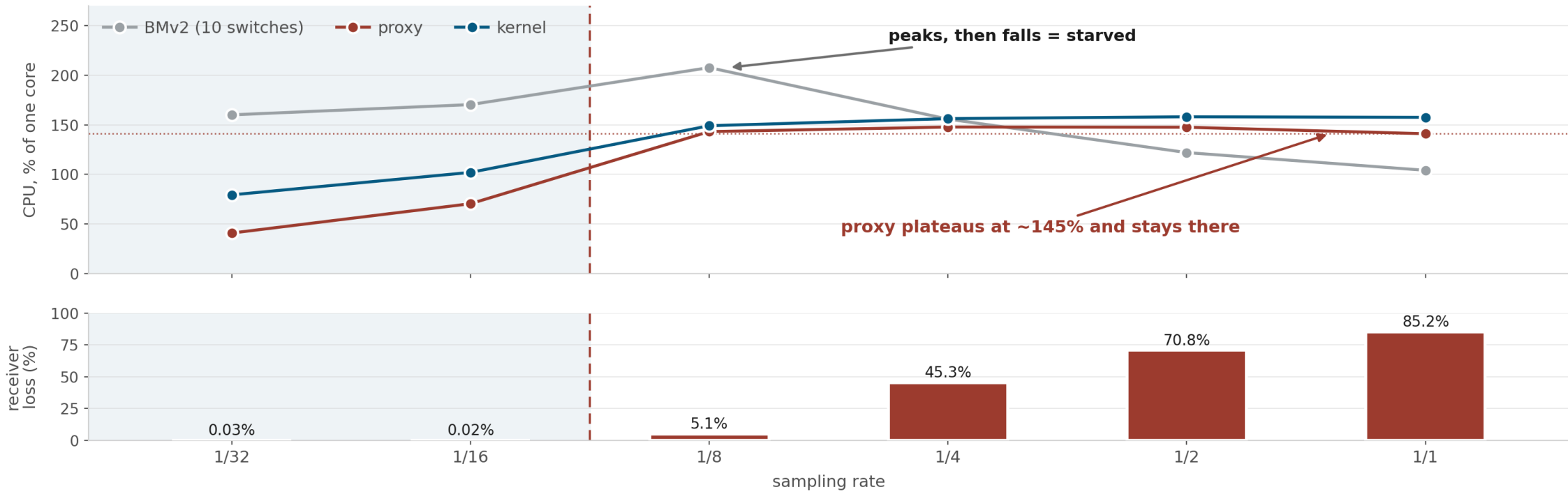
Above 1-in-16, sampling stops buying anything and starts costing throughput

Sampling 16x harder returns the same number of samples — while the receiver loses 85% of the flow



The wall is the proxy — and BMv2 is being starved, not saturated

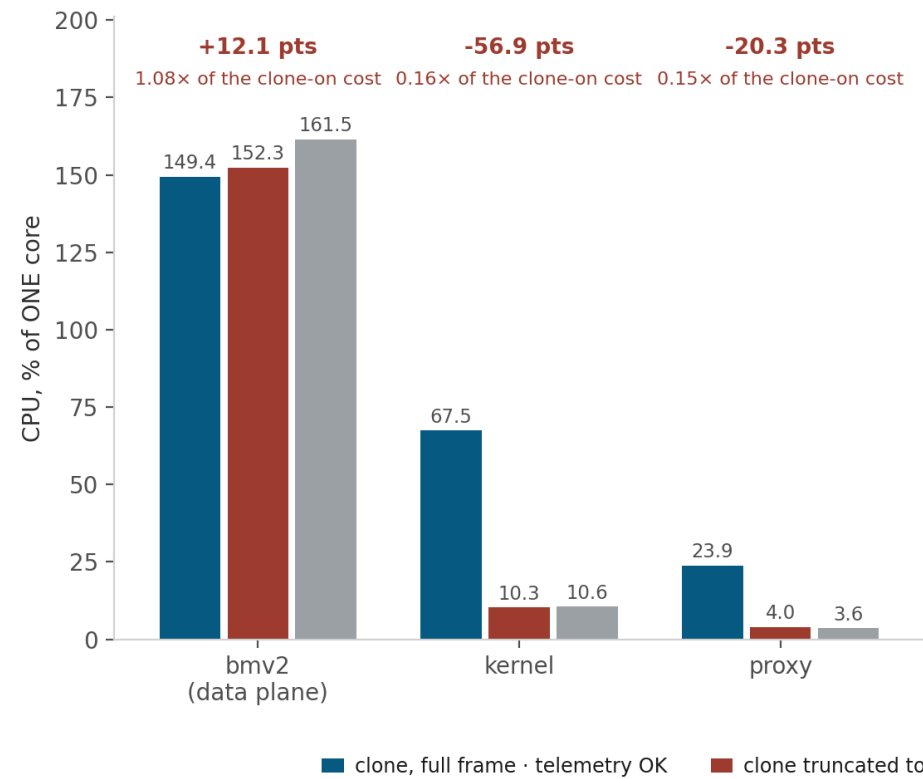
The proxy flattens at ~145% of one core exactly where loss begins; BMv2 peaks and then falls; the machine never uses 5 of its 14 cores



The sampling bill lands on the kernel and the proxy

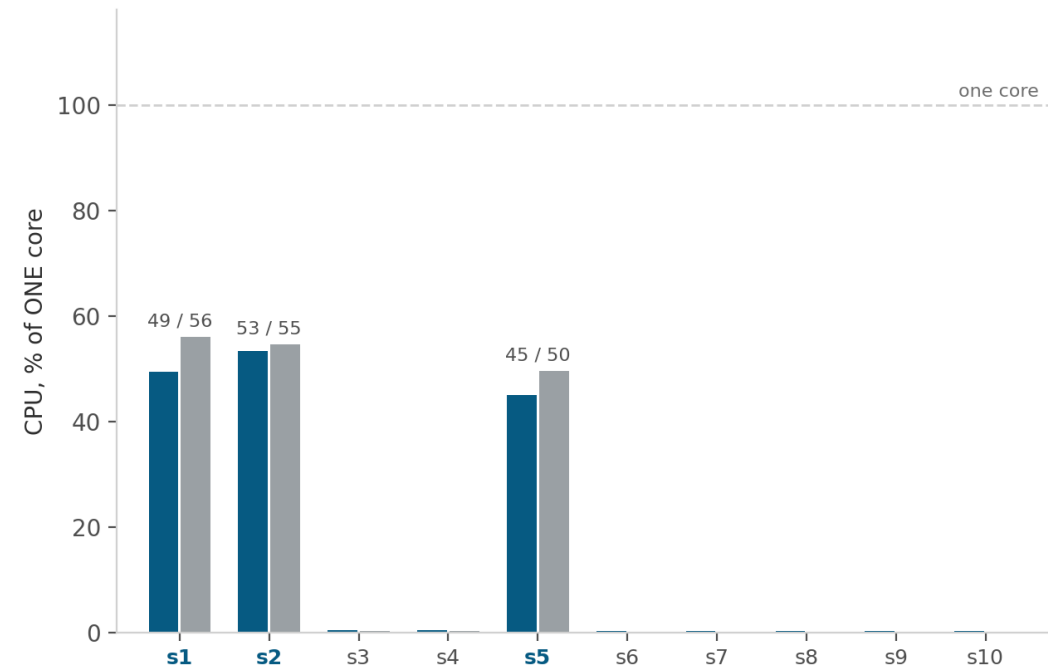
Turning sampling off, at 4x production load

HIGHER with sampling off — that difference is noise, not a saving.



The same total, opened up: 3 of 10 switches do the work

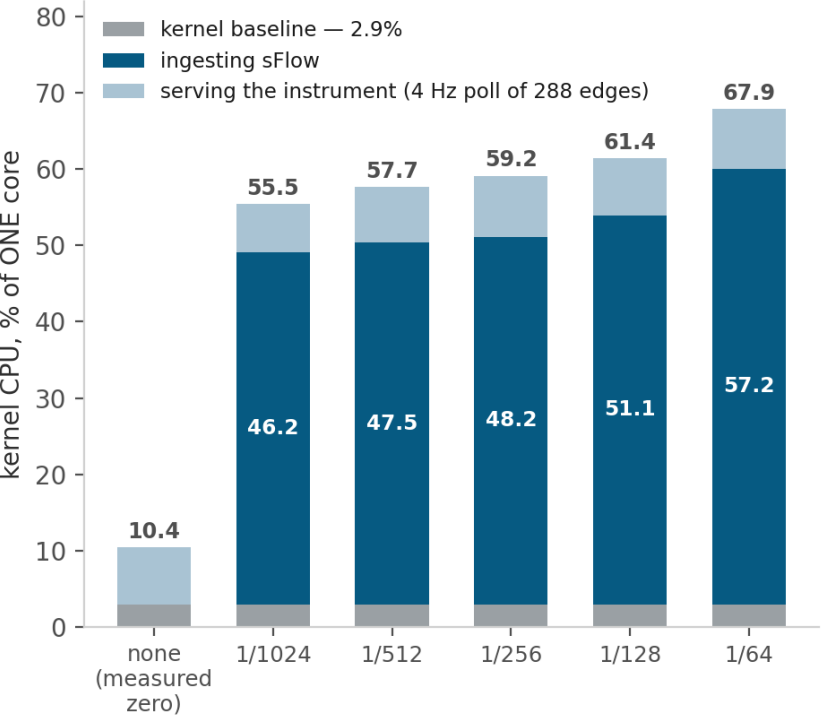
s1, s2, s5 are the switches the flow crosses (confirmed on the tx counters); the other 7 sit at 0.4%. So "bmv2 at 150% of a core" is really three single-threaded switches at half a core each — far from a ceiling.



206  $\mu$ s of CPU per sample, inside the range measured

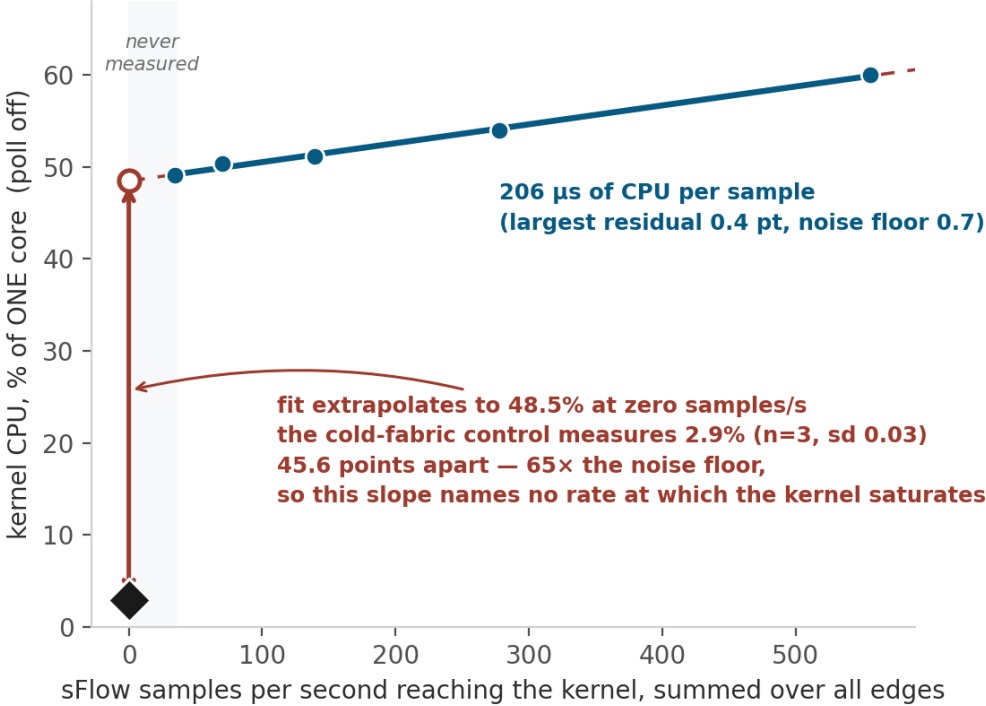
Where the kernel's CPU goes, by sampling rate

The instrument is a flat 6.3–8.0 points at every rate, so subtracting it is safe — and it is not what the kernel's CPU is spent on.



The marginal cost is linear — the line is not

Five cells across a 16 $\times$  range fit a straight line to within the noise floor. Extended to zero that same line overshoots the measured zero by 17 $\times$ , so the slope may not be extrapolated: it gives no rate at which the kernel saturates



# The 45-point thread is not sampling cost

Named from the kernel log of the same boot, not from a thread-id offset — 32× more samples move it by 1%

| thread                        | 1/1024 | 1/32   | what it is             |
|-------------------------------|--------|--------|------------------------|
| calFlowPathByQueried          | 46.31% | 46.84% | a 1 kHz path recompute |
| run                           | 3.98%  | 13.86% | the sFlow ingest       |
| testCalAvgFlowSendingRates... | 0.00%  | 0.00%  | never runs             |
| purgeIdleFlows                | 0.00%  | 0.00%  | never runs             |

## Why the threads could be named at all

Both the CPU trace and the kernel log come from the same boot. The earlier round archived no log, and the thread-id offset moves between builds — so those traces can never be mapped back to names.

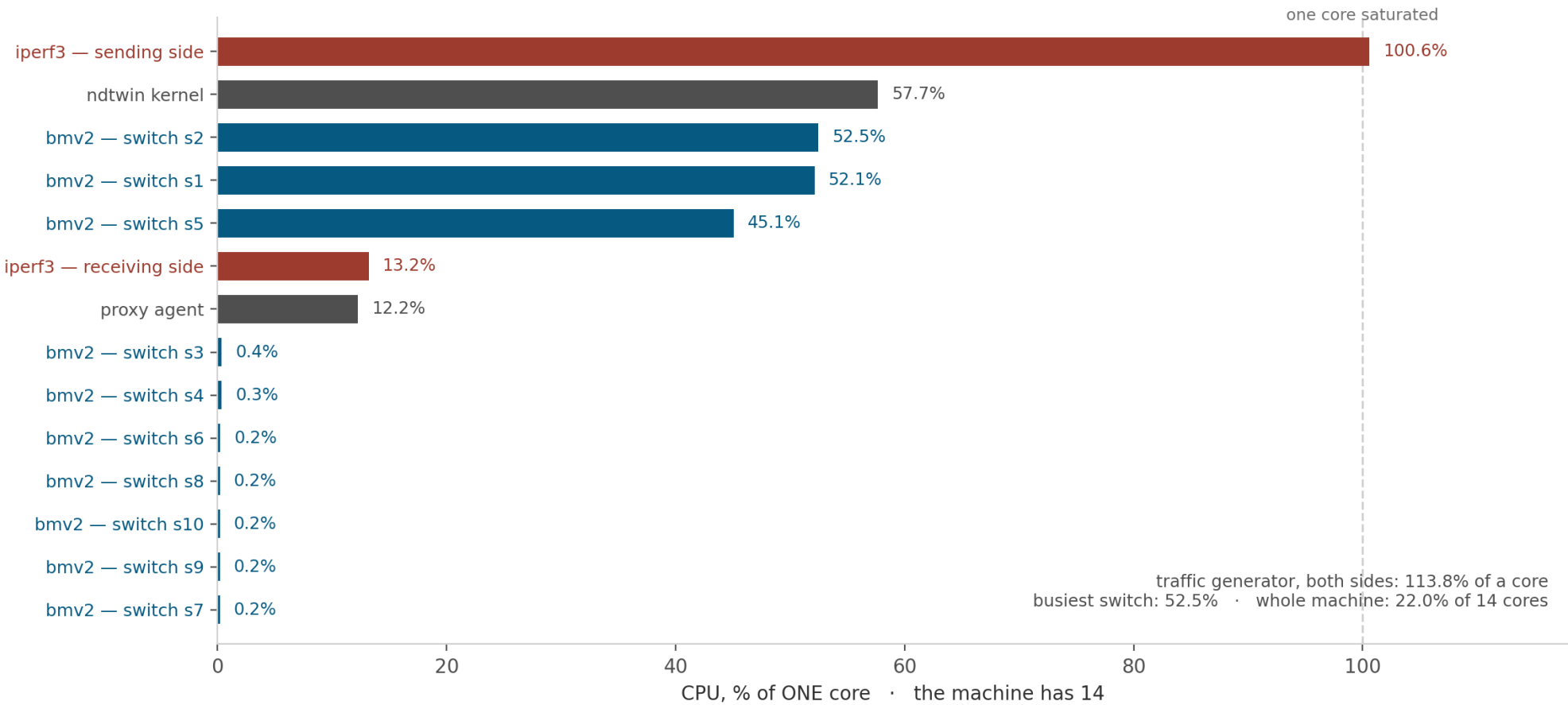
### 1 Thirty-two times the samples moves it by one percent

So those 45 points have nothing to do with sampling. It is a full path recompute running at 1 kHz — and it is inherited: present at 28b8b13.

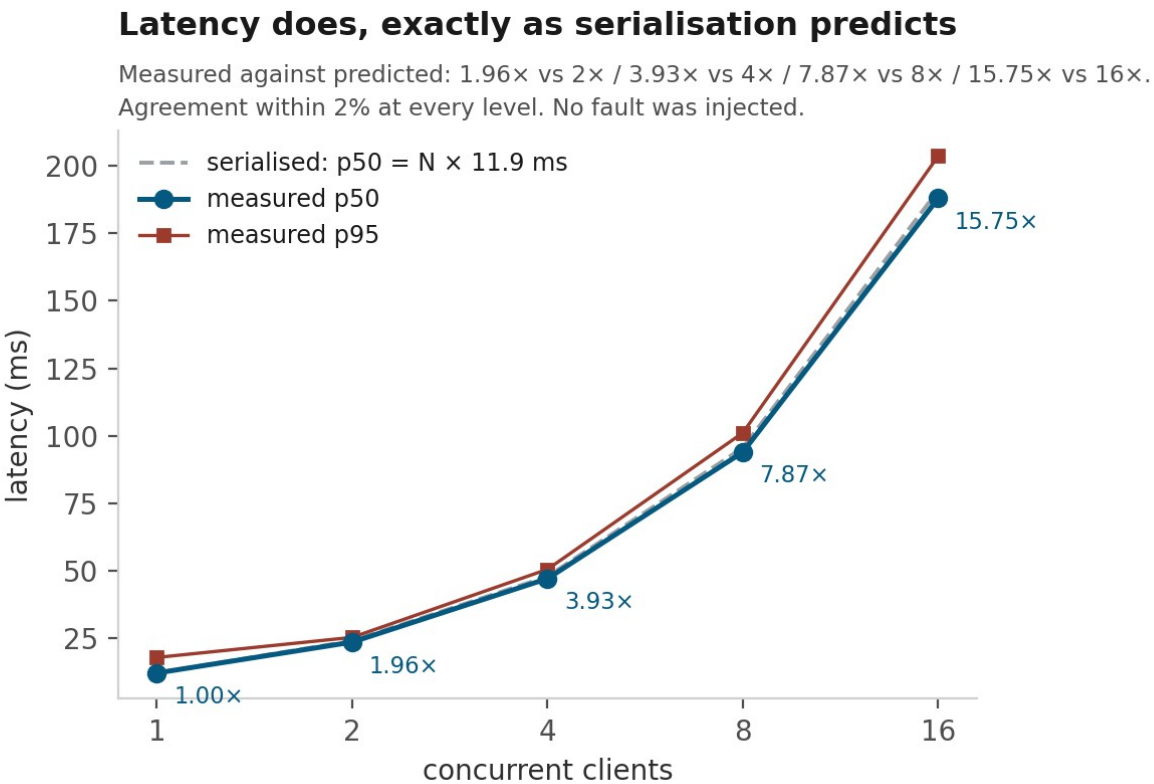
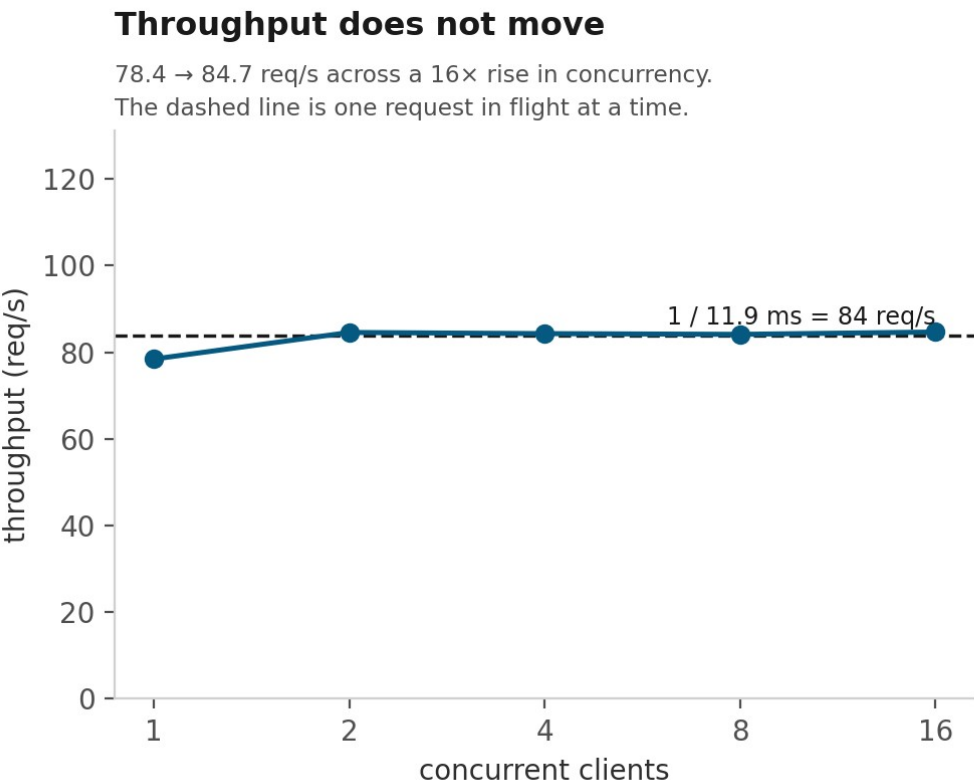
### 2 The ingest thread is the one that scales

3.98% → 13.86%, a factor of 3.48 as the sample rate rises. That is what sampling actually costs inside the kernel.

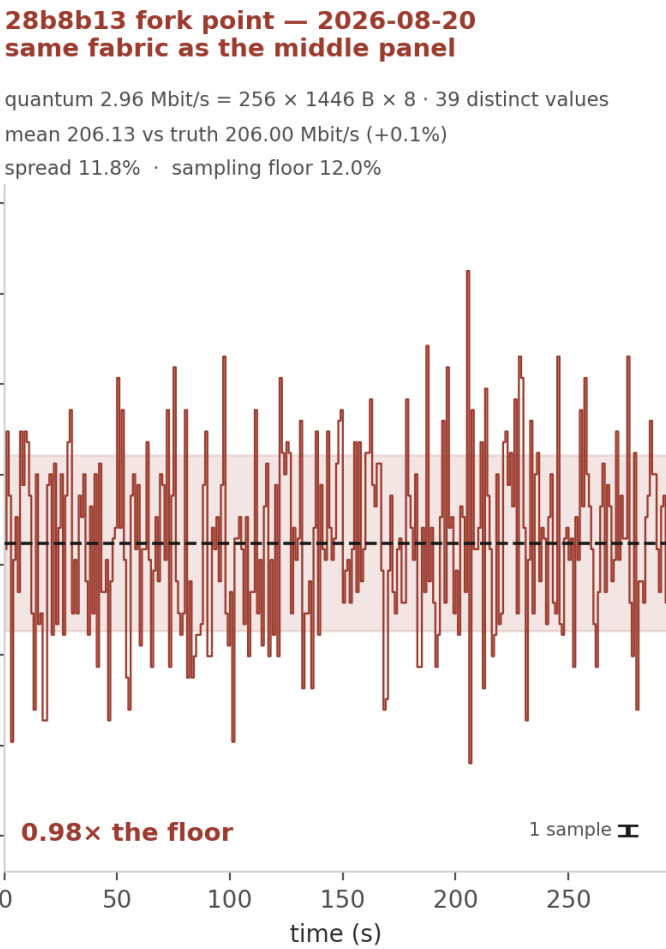
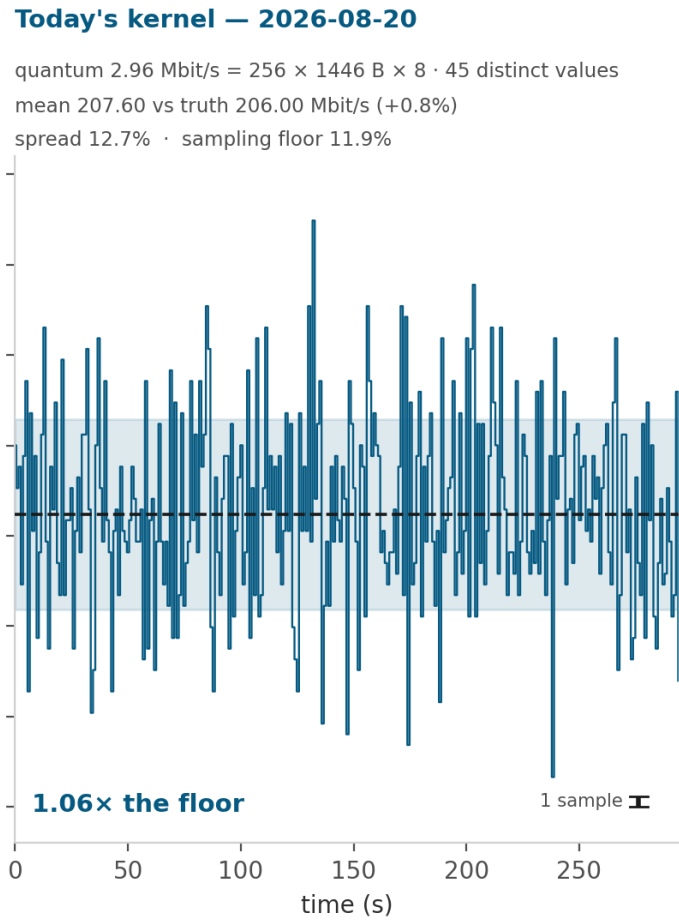
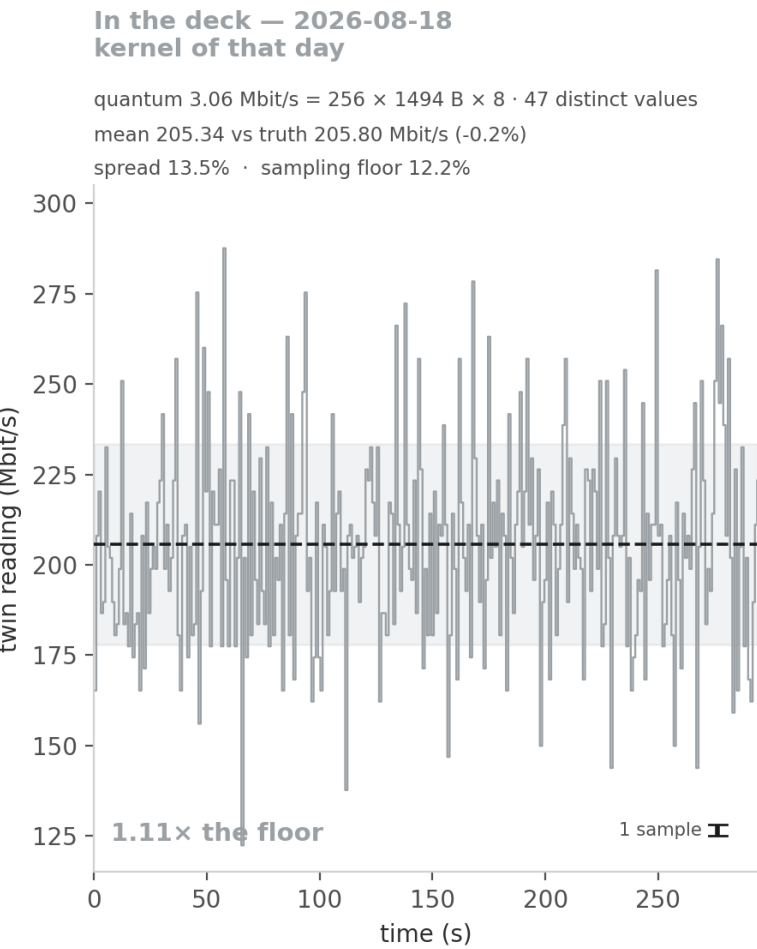
The biggest CPU consumer on the box is the instrument, not the fabric



The northbound API serves one request at a time



The staircase, and its jitter, predate the fork



# Under concurrency, the twin reads high

Every 128-host measurement until now carried one flow and loaded three switches in ten — this one loads all ten

| edge class      | as measured  | recounted | edges             |
|-----------------|--------------|-----------|-------------------|
| host → switch   | 1.23×        | 1.26×     | 62 of 122         |
| switch → host   | 1.34×        | 1.41×     | 62 of 128         |
| switch → switch | 1.26×        | 1.33×     | 8 of 32           |
| OVS, 64 flows   | flat to 0.2% | —         | all three classes |

## 1 High on every edge class, well past the noise

The aggregate sampling error floor here is 1.7–2.9%. Twenty-plus points is not it — and four rounds all point the same way.

## 2 Recounted from the raw with a different method

Different windows, an independently written interface mapping, no filtering of edges: same direction, same size, same ordering of the three classes.

### Why a single ratio is not quoted

Sixteen flows, same settings, two fabric generations: 1.19 on one and 1.03–1.05 on the other. Within one generation the repeats agree to four decimals, so that 14% is not run-to-run noise — it is unexplained. Ratios from different generations are never divided.

### What this does not show

Not that scale is the cause. The traffic generator changed at the same time as the host count, so the two are confounded; separating them needs a four-host run with the new generator, which has not been done. The mechanism is unidentified — the shared accounting path is the candidate, on the strength of both planes behaving alike.

# A result we pre-registered, and then withdrew

Six cells said the same sample count does not mean the same precision — the control says we cannot tell them apart

---

## 1 What the six cells showed

Two levels, each reached from opposite directions. The pairing worked — sample counts matched to 0.84% and 0.44% — and every cell was healthy.

## 2 What was about to be claimed

That two cells with the same sample count are not equivalent, so the whole matrix has to be run. It rested on every pair clearing 0.1, and one pair cleared it by 0.040.

## 3 What the control did to it

Same parameters three times, a full fabric rebuild between each: the spread moved 0.136 on its own, and across the three runs by 0.240. The threshold cannot separate cell from cell.

### The rule that made this decidable

The threshold was written down before the run. Afterwards there was a reading that would have rescued the claim — widen it to 0.24 — and taking it is exactly what pre-registration exists to forbid. That is next round's hypothesis, with its own registration.

### Why this page is here at all

The reversal had already been written into a draft of this deck. It came out because the control came back — which is worth more than the six cells were.

# A boot deadlock, and a same-boot A/B

Six of ten boots never converged; the dump names two queues full at the same time — mechanism, fix, and the control

## 1 The dump names the mechanism

Two controller event queues full at once, 12 and 43 senders blocked. The damage is not the stall — it is the kernel's topology poll starving behind it.

## 2 The fix gives two unbounded waits a deadline

Both inside the handler. The evidence required to call a link dead is untouched. A second fix moves the work out of the loop entirely.

## 3 The control is a same-boot A/B, within one hour

Same machine, same defaults, alternating arms — drift is bounded by an hour, not a day.

| arm         | stuck | fidelity | invariant |
|-------------|-------|----------|-----------|
| no fix      | 2/3   | 1/3      | violated  |
| deadline    | 0/3   | 3/3      | violated  |
| out of loop | 0/3   | 3/3      | holds     |

### The key cell is not the zero

It is the run where the block did happen — 28 timeouts — and the poll came back on its own after 148 s. That is the fix working, not the fault being absent.

### What N = 3 does not buy

At the old rate, three clean runs happen 3.7% of the time by chance. A first verification, not a conclusion.

## SECTION 2

# Engineering, and what's next

Bring-up as one command, truncation and batching, and what the next report will carry

---

# One command brings the lab up and checks it

Two data planes start in opposite orders and neither fails loudly when reversed — so this page shows what bring-up now enforces

|     | starts first      | and reversing it looks like                                                                    |
|-----|-------------------|------------------------------------------------------------------------------------------------|
| OVS | the control plane | switches dial a dead port; no log line mentions the port, only a topology that never converges |
| P4  | the data plane    | the proxy's first RPC is refused and it exits before opening its own port                      |

## 1 One number decides everything downstream

Fabric size and the kernel's model both come from the host count. A 128-host fabric beside a 4-host model happened once, silently.

## 2 Bring-up ends by sending a real packet

Not by reading a status page. Ten green switches over a network carrying nothing has happened twice.

## 3 Shutdown proves what it killed

Process start time against the pidfile's write time. Names and argv are useless — every service is exec'd through a wrapper.

### Why this belongs in an experiment talk

Before it, every number carried an unchecked assumption: that the testbed was the shape the operator believed. This is that assumption becoming enforced.

### No boot times here, on purpose

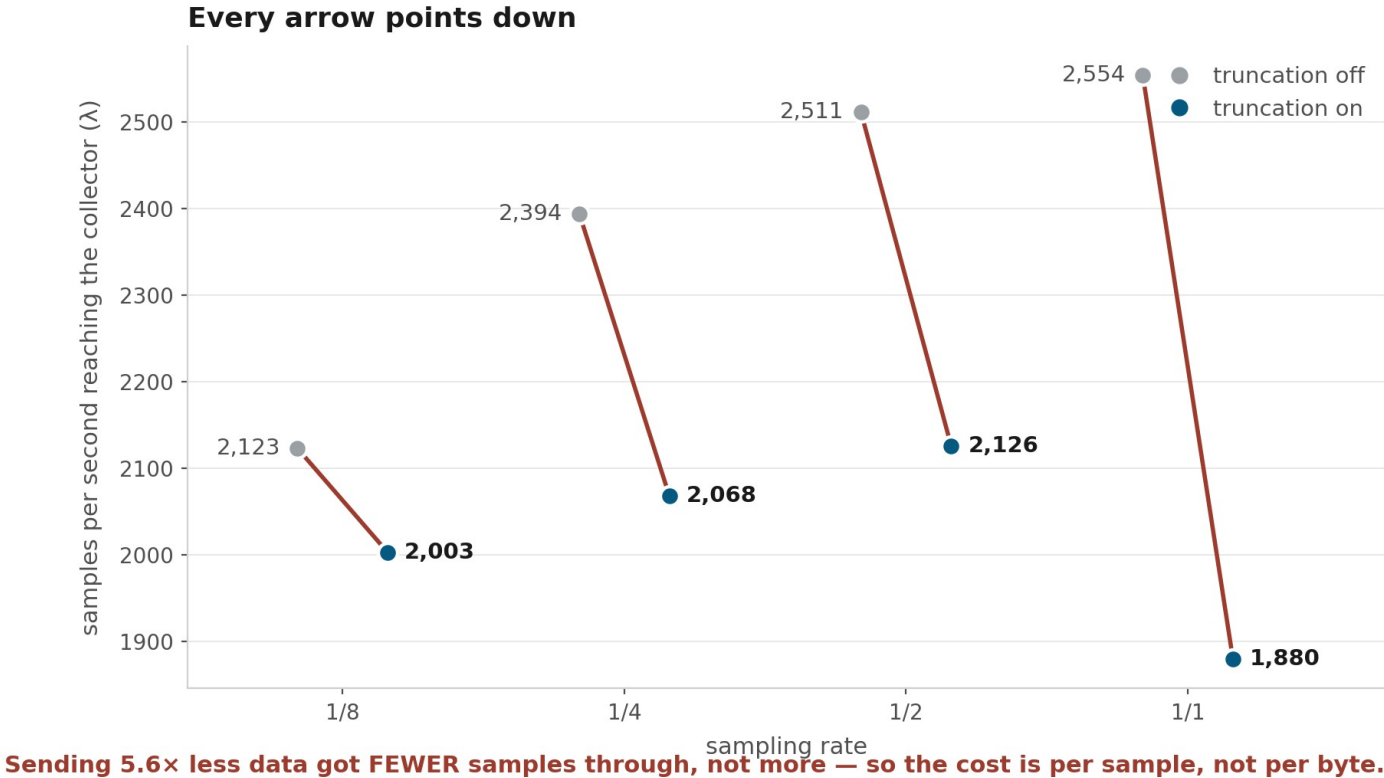
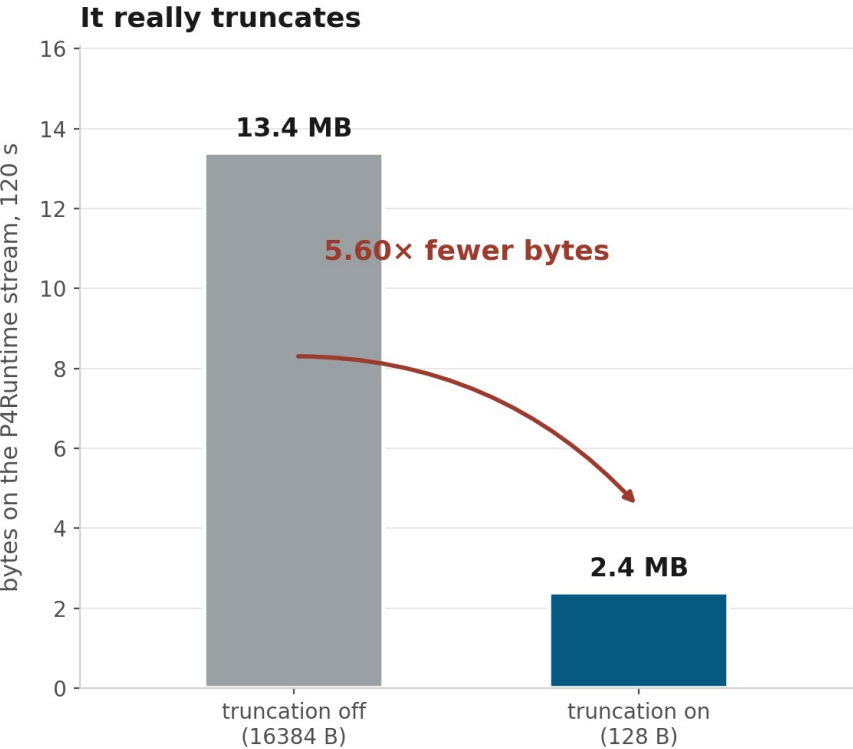
Ten boots at defaults, six never converged. “How long a successful boot takes” is a floor, not a number — and that defect is the previous page.

### Stated as unfinished

The claim is a convention, not a lock — it stops the tool's own verbs, not a bare command. Cleanup does not assert what it removed.

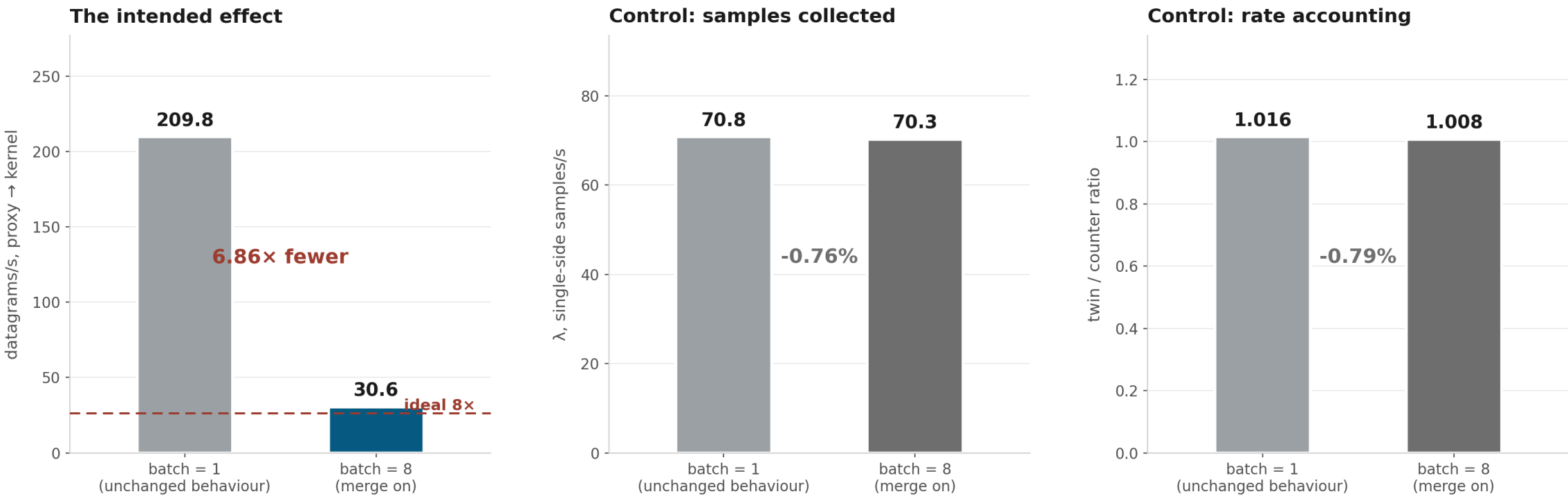
### Truncation works, and buys nothing

Sampled copies cut 5.6x on the P4Runtime stream — and every rate above the ceiling got slightly worse, not better



### Merge merges — and nothing else moved

One cell at 1-in-256, two arms, one value changed (NDTWIN\_SFLOW\_BATCH 1 → 8). Same binary, quantum identical to the bit.



A merge that moved either control would be an implementation bug, not a working merge — that reading was fixed before the run. **Effect on the sampling ceiling: NOT MEASURED.**

# Truncation and batching, after the experiments

One cut the bytes and bought nothing; the other cuts the datagrams and has not been asked the question yet

---

## 1 Truncation works, and buys nothing

5.6× fewer gRPC bytes, nothing silent — 35 distinct readings against the control's 32. Yet all four rates move the wrong way. Per-byte was the registered test, and it is out.

## 2 Merge is wired, and it merges

One variable, batch 1 against 8: datagrams fall 209.8/s → 30.6/s, while the sample count moves 0.8% and the ratio 1.016 → 1.008. Both of those staying flat was the gate.

## 3 Whether it moves the wall is the open question

The gate proves it can be done, not that it helps. If the cost sits on the send side the wall moves; if it sits on the receive side it will not, and the next cut is receive-side concurrency.

### The switch-side knob, for contrast

Truncating the clone inside the switch reads zero on every edge and reports no error at all. Not an optimisation — an outage. That is why emitter-side truncation is a measured necessity rather than a preference.

### 6.87×, not 8×

518 extra datagrams, which is what a 0.2 s age timer flushing part-full batches looks like.

### One side effect, recorded not explained

Distinct readings 34 → 14. The total is unchanged; the time grain is not.

# Planned for the next report

Three again, in the same format — each with the number it starts from and the test that decides it

---

1

## Take the boot-ring result from suggestive to conclusive

0/3 against 2/3, and the second fix holds the invariant outright — 159 stack samples, no violation. Three clean runs happen by chance 3.7% of the time.

Done when the arms reach 0/6 to 0/8 and the boot-success rate is re-measured on the fixed build.

2

## Finish the third question on its own terms

The binaries can be told apart, which was the prerequisite. Swapping the switch binary changes the timing every layer depends on.

Done when the whole L0-L4 suite passes on the fast build — the test written down on 20 August.

3

## Ask whether batching moves the ceiling

Merge is wired and through its gate: 6.87× fewer datagrams with the sample count and the ratio both flat. What that buys at the wall is the part nobody has measured.

Done when the 1-in-8 cell — the first rate that loses packets — is run with batching on and off, and the loss either moves or does not.

# Where it stands

What is settled, what is run but not finished, and what is still unknown

## Settled

### The 128-host failover budget balances

Detection is 87% of it, and the cause is arithmetic: hosts stretch the gap between two probes of one port.

### Detection is 3.9× faster, threshold untouched

One constant. Idle false-positive rate zero across  $\sim 10^5$  probe opportunities.

### A sample has a price — on the kernel and the proxy

206  $\mu$ s per sample inside the range run; flat on the switches across a 4× change in rate.

### Sampling harder stops paying at 1-in-32

Above 1-in-16 it costs throughput instead — 85% of the flow lost at 1-in-1.

### Truncation is not the fix

5.6× fewer bytes moved the ceiling by nothing. The cost is not per byte.

### Under concurrency the twin reads high

Four rounds, three edge classes, and an independent recount agrees. How high is not settled.

## Open, and stated as open

|                         |                                                                     |
|-------------------------|---------------------------------------------------------------------|
| Boot convergence        | six of ten boots at defaults; fix verified at N = 3 only            |
| Fast build as default   | the L0–L4 acceptance run is scheduled, not done                     |
| Loaded false positives  | the idle rate is zero; under load it is unmeasured                  |
| Scale × concurrency     | the direction holds; the size moves 14% between fabric generations  |
| The intercept           | 206 $\mu$ s/sample is a marginal cost inside a range, not a ceiling |
| The ceiling's mechanism | receive-side, per sample — inferred, not identified                 |
| Batching (merge)        | wired and gated; what it does to the wall is unmeasured             |
| Northbound concurrency  | one lane, measured; not yet a problem, not yet addressed            |
| Residual in the budget  | 3.6 s across noise and an epoch difference — not explained          |

The three marked items are what the next report answers; the rest are in doc/KNOWN-ISSUES.md.